МОНГОЛ УЛСЫН ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ

KOICA
Korea International Cooperation Agency

Хиймэл оюуны үндэс

Лекц № 3

This e-learning content was developed by the support of KOICA

МОНГОЛ УЛСЫН ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ

KOICA
Korea International Cooperation Agency

Хичээл 3.3
Хиймэл оюун ухааны хэрэгслүүд

Компьютерын ухааны салбар

This e-learning content was developed by the support of KOICA

Энэ удаагийн лекцээр

Хиймэл оюун ухаанд өргөн хэрэглэгддэг хийсвэр өгөгдлийн төрлүүдийн онцлог, тэдгээрийг программчлалд ашиглах хэрэгслүүд болон нээлттэй эхийн санг ашиглах зарчим, хэрэглээ зэргийг судлах болно. Үүнд:

- Объект хандалтат программчлал;
- Өгөгдлийн бүтэц;
- TensorFlow;
- Дүгнэлт.

Ж.Алимаа. ШУТИС, МХТС, Компьютерийн ухааны салбар, дэд профессор (PhDДоктор). (e-mail: alimaaj@must.edu.mn)



Суралцахуйн үр дүн

Хиймэл оюун ухааны хэрэгслүүдийн нэг болох Python программчлалын хэлний дараахь ойлголтуудыг судалж тэдгээрийг практикт хэрэглэх жишээтэй танилцах болно. Үүнд:

- Объект хандалтат программчлалын талаарх ойлголт;
- Өгөгдлийн бүтцэд багтах төрлүүд, тэдгээрийн онцлог, хэрэглэх зарчимууд, тэдгээрийг хэрэглэх боломжууд;
- Хиймэл оюун ухаан, машин сургалтанд зориулсан нээлттэй эхийн программ хангамжийн сан TensorFlow ашиглах .

Объект хандалтат программчлал

Объект хандалтат программчлал (ООР) нь объектын тухай ойлголт дээр суурилсан програм хангамж боловсруулах арга зүй юм.

Python бол объект хандалтат программчлалын хэл юм.

Класс тодорхойлох

Класс- бодит ертөнц дэх асуудлыг шийдэхэд хэрэгтэй шинэ төрөл юм. Классын бүрэлдэхүүнд өгөгдлийн атрибут/шинж чанар ба метод/арга багтана.

Атрибут-классын шинж чанаруудыг дүрслэн харуулдаг.

Метод-классын үйл ажиллагааг хэрэгжүүлэх буюу тодорхой үүрэг гүйцэтгэнэ.

Python хэлэнд класс тодорхойлох бичиглэлийн хэлбэр :

```
Class nameofclass(SuperClass)
  _init_        # __init__() функц нь шинэ объект үүсгэхэд class ашиглах бүрт автоматаар дуудагддаг.
  attributes
  methods
```

Объект хандалтат программчлал

Класс дотор Объект үүсгэх

Объект – нь өгөгдөл ба функцыг хоёуланг нь агуулсан хийсвэр өгөгдлийн төрлөөс үүснэ.

Объект – г өгөгдлийн атрибутууд ба түүн дээр үйлдэл гүйцэтгэдэг методуудаас бүрдэх бие даасан нэгж гэж тодорхойлж болно.

Классын нэг тохиолдол юм. Үүнийг үүсгэхдээ классын нэрийг ашиглана.



Python хэлэнд объект үүсгэх бичиглэлийн хэлбэр :

Class nameofclass(SuperClass)

init # __init__() функц нь шинэ объект үүсгэхэд class ашиглах бүрт автоматаар дуудагддаг классын байгуулагч функц юм.

attributes

methods

Жишээ класс:

```
class Student(object):
    def message(self):
        print("Сайн байна уу. Би – оюутан")
st1=Student()
st1.message() # Классын метод руу хандаж байна.
```

Объект хандалтат программчлал

Удамшил

Нэг класс (дэд анги) өөр классын (эцэг эх класс) шинж чанар, аргуудыг өвлөн авах боломжийг олгодог механизмыг **удамшил (Inheritance)** гэнэ.

ОХП дахь удамшил нь бусад классуудад ашиглаж болох талбар, шинж чанар, method-уудтай/аргуудтай/ класс үүсгэх боломжийг олгодог. ОХП хэллэгээр үндсэн классыг удамшсан (эцэг, дээд класс), түүнээс үүссэн классыг удамшлын (дэд класс, удамшлын гаралтай) гэж нэрлэдэг.

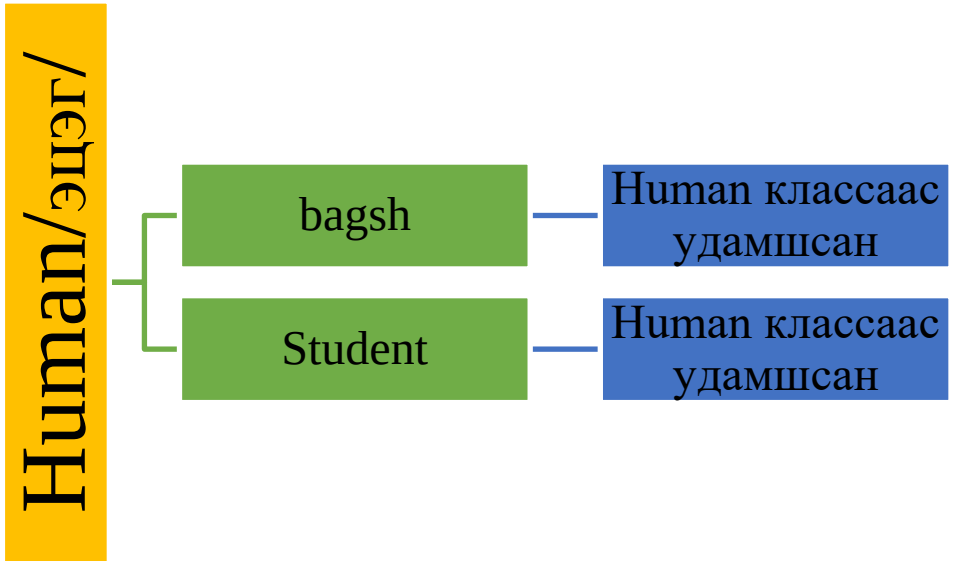
Хүү класс эцэг классаас удамшихдаа дараахь бичиглэлийн хэлбэртэй байна :

```
Class SubclassName(BaseClassName)
  class body
```

Объект хандалтат программчлал

Удамшил хэрэгжүүлэх жишээ: Human классаас удамших багш, оюутан классууд үүсгэх

```
class Human():
      name=""
      def __init__(self): #Үндсэн классыг бүтээгч
          print("Хүн гэдэг класс байгуулав")
#----------------------------------------
class bagsh(Human):
      tenhim="" #Тэнхимийн нэр
      alban_tushaal="" #Албан тушаал
      def __init__(self): #Хүү классыг бүтээгч
          print("Багш гэдэг класс байгуулав")
class Student(Human):
      salbar="" #Салбарын нэр
      grupp="" #группын дугаар
      def __init__(self): #Хүү классыг бүтээгч
          print("Оюутан гэдэг класс байгуулав")
#------{Классуудын жишээ үүсгэх -------
h=Human()
pr=bagsh()
st=Student()
```



Үр дүн:

```
Хүн гэдэг класс байгуулав
Багш гэдэг класс байгуулав
Оюутан гэдэг класс байгуулав
```

Объект хандалтат программчлал

```
1  class Human():
2      def __init__(self,name): #Үндсэн классыг бүтээгч
3          self.name=name
4          print("Хүн гэдэг класс байгуулав")
5      def info(self):
6          print(self.name,end="")
7  #----------------------------------------
8  class bagsh(Human):
9      def __init__(self,name,tenhim,alban_tushaal): #Хүү классыг бүтээгч
10         self.name=name
11         self.tenhim=tenhim #Тэнхимийн нэр
12         self.alban_tushaal=alban_tushaal #Албан тушаал
13         print('Багш гэдэг класс байгуулав: ')
14     def info(self):
15         Human.info(self)
16         print('Тэнхим: "{0}"Албан тушаал: "{1}"'.format(self.tenhim,self.alban_tushaal))
17 #----------------------------------------
18 class Student(Human):
19     def __init__(self,name,salbar,grupp): #Хүү классыг бүтээгч
20          self.name=name
21          self.salbar=salbar #Салбарын нэр
22          self.grupp=grupp #группын нэр дугаар
23          print('Оюутан гэдэг класс байгуулав')
24     def info(self):
25         Human.info(self)
26         print(' Салбар: "{0}"  Групп: "{1}"'.format(self.salbar,self.grupp))
27 #----------------------------------------
28 pr=bagsh('Батаа ','Компьютерийн Ухаан ','Профессор')
29 pr.info()
30 print()
31 st=Student('Энх  ',' ПХИ-4 ',' Эчнээ-2-р групп ')
32 st.info()
```

Удамшил хэрэгжүүлэх жишээ: Human классаас удамших багш, оюутан классууд үүсгэх

Үр дүн:

```
Багш гэдэг класс байгуулав:
Батаа Тэнхим: "Компьютерийн Ухаан "Албан тушаал: "Профессор"

Оюутан гэдэг класс байгуулав
Энх   Салбар: " ПХИ-4 "  Групп: " Эчнээ-2-р групп "
```

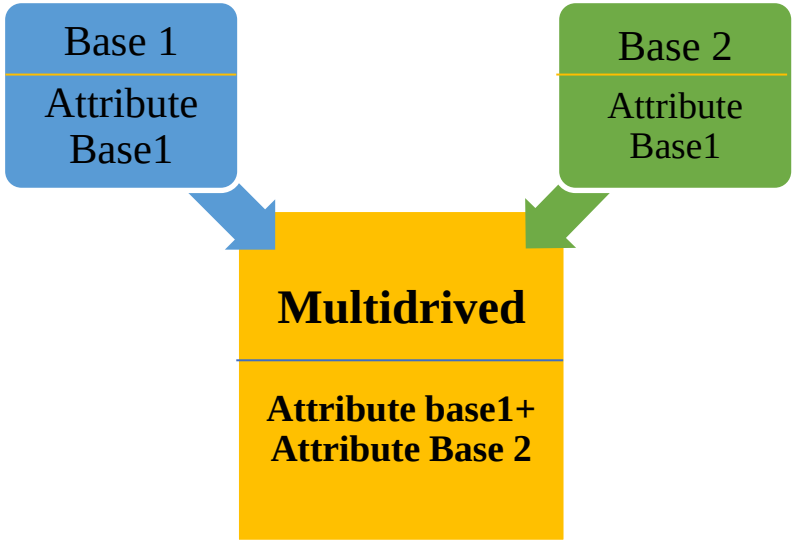
Объект хандалтат программчлал

Нийлмэл удамшил

Python хэл нь нэг болон түүнээс дээш тооны классаас удамших боломжтой нийлмэл удамшлыг дэмждэг.

Нийлмэл удамшлын үед дэд класс нь эцэг классуудынхаа бүх шинж чанарыг өвлөн авдаг.

Нийлмэл удамшлын синтакс нь ердийн удамшлын синтакстай маш төстэй юм.



Нийлмэл удамшил үүсгэх бичиглэлийн хэлбэр :

Class **SubClassName**(BaseClassName1, BaseClassName2,.. BaseClassNameN)

class body

Өгөгдлийн бүтэц

Өгөгдлийн хийсвэр төрөл

Python бол олон талт, хүчирхэг программчлалын хэл бөгөөд түүний давуу талуудын нэг нь хийсвэр өгөгдлийн төрлүүдийг (ХӨТ/ADT) дэмждэг.

Python хэл нь класс болон объект хандалтат программчлалыг ашиглан хэрэглэгчийн хийсвэр өгөгдлийн төрөл (Abstract Data Type - ADT/ХӨТ) үүсгэх боломжийг олгодог.

Хийсвэр өгөгдлийн төрөл ба Хэрэглэгчийн тодорхойлсон өгөгдлийн төрлийн ялгаа

	Хийсвэр өгөгдлийн төрөл	**Хэрэглэгчийн тодорхойлсон өгөгдлийн төрөл**
Гол төвлөрөл	Өгөгдөл ба үйлдлийн хийсвэрлэл. Ямар үйлдлүүдийг хийж болохыг тэдгээрийн зан төлвөөр тодорхойлдог.	Ямар үйлдлүүдийг хийж болох ба тэдгээрийн зан төлөв ямар байх вэ? Одоо байгаа анхдагч төрлүүдийг нэгтгэх эсвэл өргөтгөх зарчмаар хэрэглэгчийн тодорхойлсон өгөгдлийн төрлийг программист бүтээдэг.
Зорилго	Өгөгдлийн бүтэц, функцүүдийн талаархи өндөр түвшний үзэл баримтлалыг өгөх	Анхдагч төрлүүд эсвэл бусад хэрэглэгчийн тодорхойлсон өгөгдөл дээр суурилсан тодорхой өгөгдлийн бүтцийг загварчилах.
Хэрэгжүүлэлт /дэлгэрэнгүй мэдээлэл/	Үйлдлүүд хэрхэн хэрэгжиж, өгөгдөл хэрхэн бүтэцлэгдсэнг тодорхойлдоггүй.	Бүтцийг хэрэгжүүлэхийн тулд өгөгдлийн төрлийг хэрхэн үүсгэх, зохион байгуулахыг зааж өгнө.
Хэрэглээ	Өгөгдлийн бүтцийг зохион бүтээх, боловсруулахад ашигладаг.	Хийсвэр өгөгдлийн төрлөөр тодорхойлсон хийсвэр ойлголтуудыг хэрэгжүүлдэг өгөгдлийн бүтцийг хэрэгжүүлэхэд ашигладаг.
Жишээ	List, Stack, Queue.	Structures, classes, enumerations, records.

Өгөгдлийн бүтэц

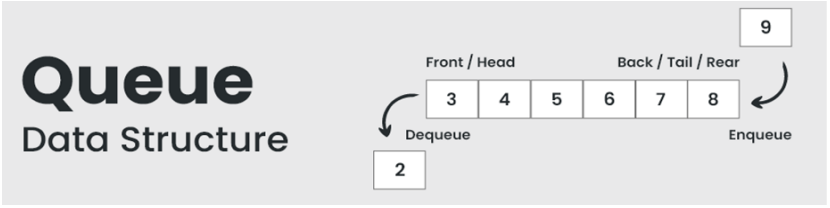
Дараалал

Дараалал гэдэг нь утгуудын цуглуулагыг хэрхэн удирдаж, хадгалахыг тодорхойлдог хийсвэр өгөгдлийн төрөл /ХӨТ/ юм.

Дарааллын ажиллах зарчим нь :

Эхэлж орсон нь түрүүлж гарах **[FIFO- First-In-First-Out]** байдаг.

Дараалал нь утгуудыг оруулсан дарааллаар нь хадгалдаг Энэ төрлийн өгөгдлийг КШУ ба ПХ-ын салбарт өргөн хэрэглэдэг.



Дарааллын үндсэн үйл ажиллагааны шинж чанар:

- Дараалал үүсгэх;
- Дарааллын урд талын элементийг харах (дарааллаас хасахгүйгээр уг элемент ямар элемент болохыг харна);
- enqueue(): дарааллын төгсгөлд элемент оруулна.
- dequeue(): Хэрэв дараалал хоосон биш бол дарааллын эхний элементийг устгаад буцаана.
- peek(): Хэрэв дараалал хоосон биш бол дарааллын элементийг арилгахгүйгээр буцаана.
- size(): Дараалалд байгаа элементүүдийн тоог буцаана.
- isEmpty(): дараалал хоосон байвал True утга буцаана; эсрэг тохиолдолд False буцаана.

Өгөгдлийн бүтэц

Дарааллыг жагсаалтаар дүрслэх

Жагсаалт нь дараалал болгон ашиглаж болох Python-н өгөгдлийн бүтэц юм.

enqueue() болон dequeue()-функцын оронд append() болон pop() функцуудыг ашигладаг.

Гэхдээ, элемент оруулах эсвэл устгахад бусад бүх элементүүдийг нэгээр нь шилжүүлэх шаардлагатай бөгөөд O(n) хугацаа шаардагддаг тул жагсаалт нэлээд удаан байдаг.

Жишээ: Хоосон дараалал үүсгэж элемент нэмж, устгаж хэвлэх

```
queue = []
queue.append('А')
queue.append('Б')
queue.append('В')
print("Анхны дараалал")
print(queue)
print("\n:Дарааллаас хассан элемент")
print(queue.pop(0))
print(queue.pop(0))
print(queue.pop(0))
print("\nЭлементийг хассаны дараах дараалал")
print(queue)
```

Үр дүн

```
Анхны дараалал
['А', 'Б', 'В']

:Дарааллаас хассан элемент
А
Б
В

Элементийг хассаны дараах дараалал
[]
```

Өгөгдлийн бүтэц

Дарааллыг классаар дүрслэх

Өгөгдлийн шинэ төрлийг үүсгэж дарааллын шинж чанараар хангаж ашигласан жагсаалтыг нуух бол өөрсдийн дараалал классуудыг тодорхойлж болно.

Үр дүн

```
Одоо  байгаа: [10, 20, 30, 40, 50]
Хасагдсан элемент: 10
Хасагдсан элемент: 20
Шинэчлэгдсэн дараалал: [30, 40, 50]
```

Жишээ: class ашиглах

```
# Дарааллын өгөгдлийн бүтцийг хэрэгжүүлэхийн тулд Queue нэртэй классыг тодорхойлно
class Queue:
    # Элемент хадгалахын тулд хоосон жагсаалт бүхий дарааллыг эхлүүлнэ
    def __init__(self):
        self.items = []
    # Дарааллын төгсгөлд элемент нэмэх
    def enqueue(self, item):
        self.items.append(item)
    # Дараалал хоосон биш бол дарааллын урд талын зүйлийг хасч,буцаана
    def dequeue(self):
        if not self.is_empty():
            return self.items.pop(0)
        else:
            raise IndexError("Cannot dequeue from an empty queue.")
    # Дараалал хоосон байгаа эсэхийг шалгана
    def is_empty(self):
        return len(self.items) == 0
# Хэрэглээний жишээ
# Queue class үүсгэх
queue = Queue()
# Дараалалд элемент нэмэх
queue.enqueue(10)
queue.enqueue(20)
queue.enqueue(30)
queue.enqueue(40)
queue.enqueue(50)
# Дараалалд байгаа элемент хэвлэх
print(“Одоо  байгаа дараалал:", queue.items)
# Дарааллын эхэнд байгаа элементийг дарааллаас хасах ба дарааллаас хасагдсан элементийг хэвлэх
dequeued_item = queue.dequeue()
print("Хасагдсан элемент:", dequeued_item)
dequeued_item = queue.dequeue()
print("Хасагдсан элемент:", dequeued_item)
# Шинэчлэгдсэн дарааллыг хэвлэ
print("Шинэчлэгдсэн дараалал:", queue.items)
```

Өгөгдлийн бүтэц

Стек

Стек нь программ хангамж болон компьютерийн шинжлэх ухаанд өргөн хэрэглэгддэг хийсвэр өгөгдлийн төрлүүдийн нэг төрөл юм. Энэ төрлийн өгөгдлийг ихэвчлэн математикийн илэрхийлэл, синтаксийг задлан шинжлэх, завсрын үр дүнг удирдахад ашигладаг.

Онцлог нь утгуудыг оруулсан дарааллаар нь хадгална, хамгийн эхэнд орсон элемент стекийн доод талд, сүүлд орсон элемент стекийн оройд байрлана.

Стек дээр хийгдэх процессууд:

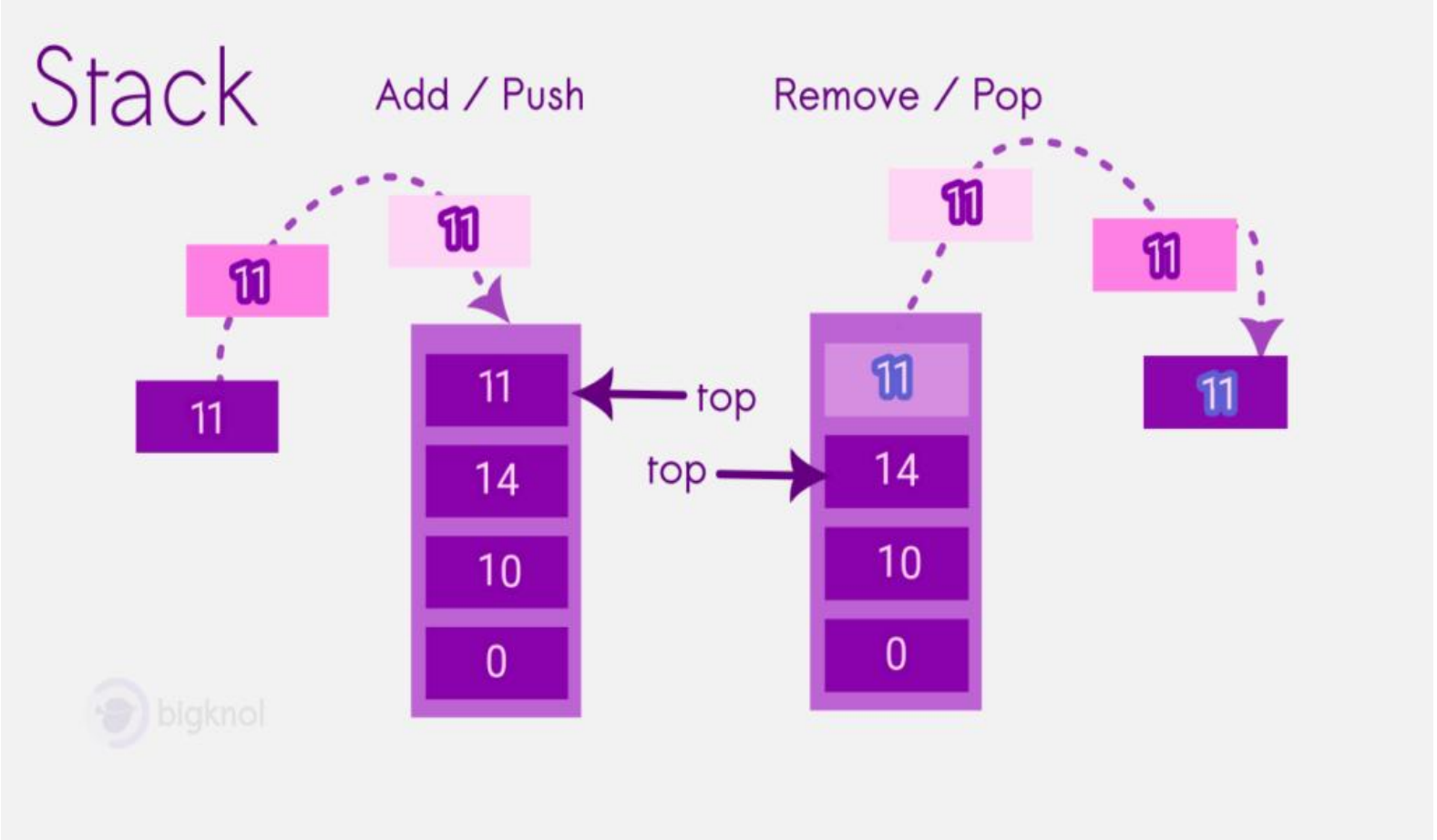
- Стек үүсгэх;
- Стекийн дээд хэсэгт элемент нэмнэ (стек рүү түлхэх гэж нэрлэдэг);
- Стекийн дээд хэсгээс элементийг гаргах (стекээс гаргах гэдэг);
- Стекийн уртыг олох;
- Стек хоосон эсэхийг шалгах;
- Стек нь тогтсон хэмжээтэй эсвэл хувьсах (өргөждөг) стек байж болно.

Өгөгдлийн бүтэц

Стекийн үйл ажиллагаа

Шинэ элементийг Стек рүү түлхэх болгонд одоо байгаа бүх элементүүдийг стекийн төгсгөл хэсэг рүү шахдаг.

Хуучин элементүүдийг харахын тулд эхлээд шинэ элементүүдийг орой дээрээс нь гаргах хэрэгтэй.



Стекийн ажиллах зарчим.

Эх сурвалж: https://bigknol.com/kotlin/stack-in-kotlin-add-and-remove-operations/

Өгөгдлийн бүтэц

Стекийг жагсаалтаар дүрслэх

List-жагсаалт нь стек хэлбэрээр ашиглахад сайн тохирох ба стек шиг ажиллахын тулд append(), pop() аргуудыг оруулж болно.

Жагсаалтын хамгийн сүүлд нэмсэн зүйл бол pop() аргаас буцаж ирэх элемент юм.

Жишээ: Стектэй ажиллах

```
stack = []
stack.append('a')
stack.append('b')
stack.append('c')
print('Одоо  байгаа стек')
print(stack)
print('\n Стекээс гарсан элементүүд:')
print(stack.pop())
print(stack.pop())
print(stack.pop())
print('\nСтекээс элементүүд гарсны дараа :')
print(stack)
```

Үр дүн

```
Одоо  байгаа стек
['a', 'b', 'c']

Стекээс гарсан элементүүд:
c
b
a

Стекээс элементүүд гарсны дараа :
[]
```

TensorFlow

TensorFlow бол хиймэл оюун ухаан болон машин сургалтанд зориулсан үнэгүй, нээлттэй эхийн программ хангамжийн сан юм. Үүнийг олон төрлийн даалгаварт ашиглаж болох боловч гүн мэдрэлийн сүлжээг сургах, дүгнэлт гаргахад ашиглагдана.

TensorFlow нь Python, JavaScript, C++, Java зэрэг олон төрлийн программчлалын хэл дээр ашиглагдах боломжтой. Энэхүү уян хатан байдал нь олон салбарт өргөн хүрээний хэрэглээг хангах боломжийг олгодог.

TensorFlow-г ашиглахын тулд эхлээд түүнийг суулгах шаардлагатай. Үүнээс гадна суулгахгүйгээр Cloud дээр Colaboratory ашиглаж болно.

TensorFlow

Жишээ: [100, 200, 300] утгатай TensorFlow тогтмолыг үүсгэгч Python программ бичих

```
import tensorflow as tf
# TensorFlow тогтмол тензор үүсгэх
# Тензорууд нь нэг төрлийн (dtype гэж нэрлэгддэг) олон хэмжээст массивууд юм.
t = tf.constant([100, 200, 300])
# Дүрс болон өгөгдлийн төрлийг хэвлэх
print("Tensor Shape:", t.shape)
print("Data Type:", t.dtype)
```

Үр дүн:

```
Tensor Shape: (3,)
Data Type: <dtype: 'int32'>
```

TensorFlow

Жишээ: Санамсаргүй утгуудаар дүүргэсэн (2, 3) хэлбэрийн хоёр TensorFlow тензорын элементийн нэмэлтийг хэрэгжүүлдэг Python программ бичих.

```
import tensorflow as tf
import numpy as np

# TensorFlow 2.x-д хүссэн гүйцэтгэлийг идэвхжүүл
tf.config.run_functions_eagerly(True)

# (2, 3) Хэлбэр бүхий хоёр санамсаргүй тензор үүсгэх
# Тензорууд нь нэг төрлийн (dtype гэж нэрлэгддэг) олон хэмжээст массивууд юм.
ts1 = tf.constant(np.random.rand(2, 3))
ts2 = tf.constant(np.random.rand(2, 3))

# Элементийн нэмэлтийг хэрэгжүүлнэ
result_tensor = tf.add(ts1, ts2)

# Анхны тензор болон үр дүнг хэвлэ
print(" Анхны тензорууд :")
print(" Тензор 1:")
print(ts1)
print(" Тензор 2:")
print(ts2)
print("\n Тензорын элементийн нэмэлтийг хэрэгжүүлсэн үр дүн:")
print(result_tensor)
```

Анхны тензорууд :

Тензор 1: tf.Tensor
([[0.33423327 0.9243049 0.13647843]
[0.66481416 0.73758093 0.76233938]],
shape=(2, 3), dtype=float64)

Тензор 2: tf.Tensor
([[0.25895318 0.44051337 0.31769159]
[0.76640689 0.15108415 0.24939268]], shape=(2,
3), dtype=float64)

Тензорын элементийн нэмэлтийг хэрэгжүүлсэн үр дүн :
tf.Tensor
([[0.59318645 1.36481827 0.45417001]
[1.43122104 0.88866508 1.01173207]], shape=(2,
3), dtype=float64)

Дүгнэлт

Энэ удаагийн лекцээр нийлмэл өгөгдлийн төрлүүд болох

- Объект хандалтат программчлалын талаарх ойлголт;
- Класс, атрибут, методын тодорхойлолт, онцлог;
- Класс дотор объект үүсгэх, удамшил түүний шинж ба хэрэглээ, нийлмэл удамшилын онцлог ба хэрэглээ;
- Хийсвэр өгөгдлийн төрөл болох дараалал, стек тэдгээрийн ялгаа, хэрэглэх боломж ;
- Хиймэл оюун ухаан болон машин сургалтанд зориулсан нээлттэй эхийн программ хангамжийн сан TensorFlow дуудах, ашиглах талаарх ойлголтуудыг өгсөн болно.

Эдгээр ойлголтуудыг лабораторийн хичээлээр туршиж үзэн бие дааж бодлого бодох болно.

Анхаарал тавьсанд баярлалаа.

Асуултаа сургалт удирдах системийн зурвасаар илгээнэ үү.